HW7 Part A: 2x2 Texture Grid Study

CS 5382 Computer Graphics | Daniel Oni | Spring 2026

Goal

The starter code displays a single copy of the webcam feed across the quad, and the image comes out upside down. For Part A, I needed to tile the feed into a 2x2 grid and make sure the image is right side up. The changes that accomplish this live in three places: the texture coordinate array, the wrap mode parameters, and one extra pixel store setting for the Y flip.

What I Changed in the Code

1. Scaled the texture coordinates from [0, 1] to [0, 2]. The vertex positions did not change. The quad still covers the canvas from (-1, -1) to (1, 1). All the tiling work happens by pushing the texture coordinates past 1.0, which causes the sampler to wrap and reuse the texture.

```
var texCoord = new Float32Array ([
    0,   0,
    0,   2.0,
    2.0, 2.0,
    2.0, 2.0,
    2.0, 0,
    0,   0
]);
```

Going from (s, t) = (0, 0) at the bottom left corner of the quad up to (0, 2) at the top left means the t coordinate crosses 1.0 halfway up the canvas. That is where the vertical wrap happens. The same thing happens on the s axis between the left and right edges, which gives the horizontal wrap.

2. Changed the wrap modes from CLAMP_TO_EDGE to REPEAT. Without this, any texture coordinate beyond 1.0 would just stretch the edge pixels. REPEAT is what actually makes the texture cycle back to 0 when the coordinate reaches 1, which is how the 2x2 pattern appears.

```
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.REPEAT);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.REPEAT);
```

Side note on this line: the starter had gl.RCLAMP_TO_EDGE for the S wrap, which is not a real WebGL constant and evaluates to undefined. That was silently falling back to whatever the default was. I cleaned it up at the same time as switching to REPEAT.

3. Added UNPACK_FLIP_Y_WEBGL to fix the upside down image. Video frame data arrives with row 0 at the top of the frame. When that gets uploaded with texImage2D by default, row 0 lands at t = 0 in the texture, which is usually treated as the bottom. The result is a vertically flipped image. Turning UNPACK_FLIP_Y_WEBGL on tells WebGL to reverse the rows during upload so that the top of the frame ends up at t = 1 where I expect it.

```
gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL, true);
```

4. Small fix to the initial texImage2D call. The starter passes the video element into texImage2D during init(), but at that moment the getUserMedia promise has not resolved yet, so the video has no frames. That triggers a warning in Chrome and can produce an INVALID_VALUE error. I replaced the source with null for the initial allocation, and the render loop fills the texture with real video data once the stream is ready.

```
gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, 400, 400, 0,
              gl.RGBA, gl.UNSIGNED_BYTE, null);
```

Mapping Diagram: Vertices to Texture Coordinates

The left side shows the quad in clip space. The vertex buffer has six entries that make up two triangles sharing the TR and BL diagonal, but the four unique corners are what matter for the mapping. The right side shows the texture coordinate space after the change. The outer red boundary is the region the sampler sees, and the dashed lines mark where s = 1 and t = 1, which is where the REPEAT wrap kicks in.

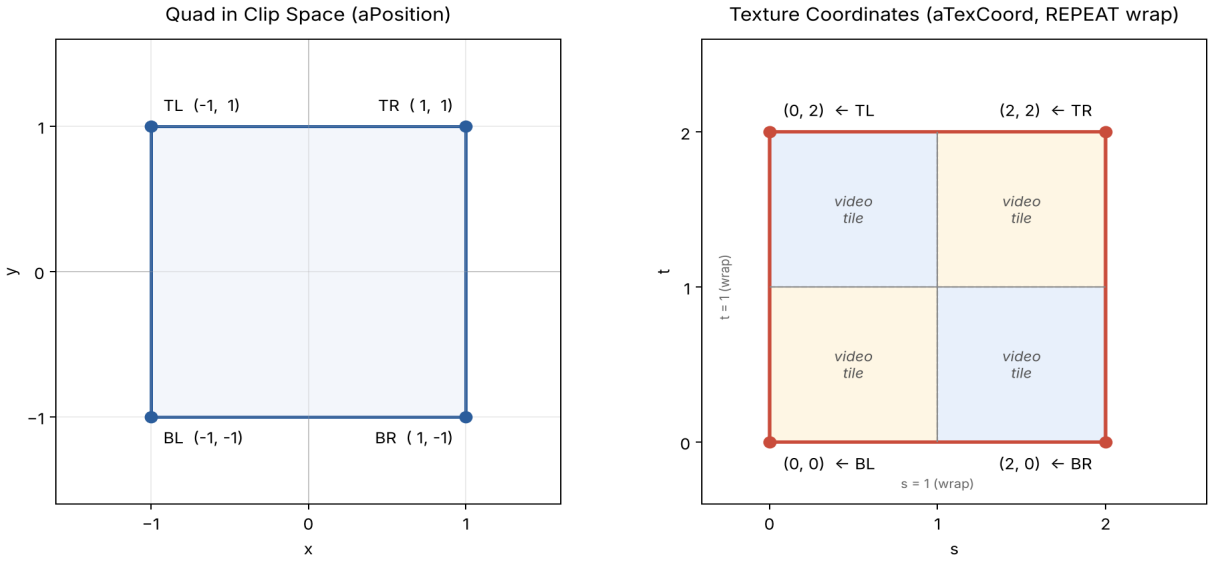


Figure 1. Each quad corner on the left maps to the labeled texture coordinate on the right. Because the texture coordinates exceed 1.0 and the wrap mode is REPEAT, the video texture is sampled four times, once for each quadrant of the canvas.

Per-corner mapping in words:

```
Vertex           Texture coord
BL (-1, -1)  ->  (0, 0)
TL (-1,  1)  ->  (0, 2)
TR ( 1,  1)  ->  (2, 2)
BR ( 1, -1)  ->  (2, 0)
```

What I Took Away From This

The main idea for me was that the tiling effect is not something the fragment shader does. The shader code is unchanged between the single copy and the 2x2 grid. All of the tiling logic is carried by two things working together: texture coordinates that reach past the [0, 1] range, and a wrap mode that turns that excess into repetition instead of clamping. If I switch the wrap mode back to CLAMP_TO_EDGE, the same texture coordinates would just smear the edge pixels across half the canvas, which is a quick way to confirm the two pieces are doing different jobs.

The UNPACK_FLIP_Y_WEBGL detail was also a good reminder that there is a separate orientation convention between image data on a web page (where row 0 is the top) and texture coordinates in GL (where t = 0 is usually the bottom). Since this assignment was specifically about texture coordinates, I was tempted to fix the flip by swapping t values in the coordinate array. Using the pixel store setting instead kept the coordinate array symmetric and easier to reason about when I was drawing the diagram.